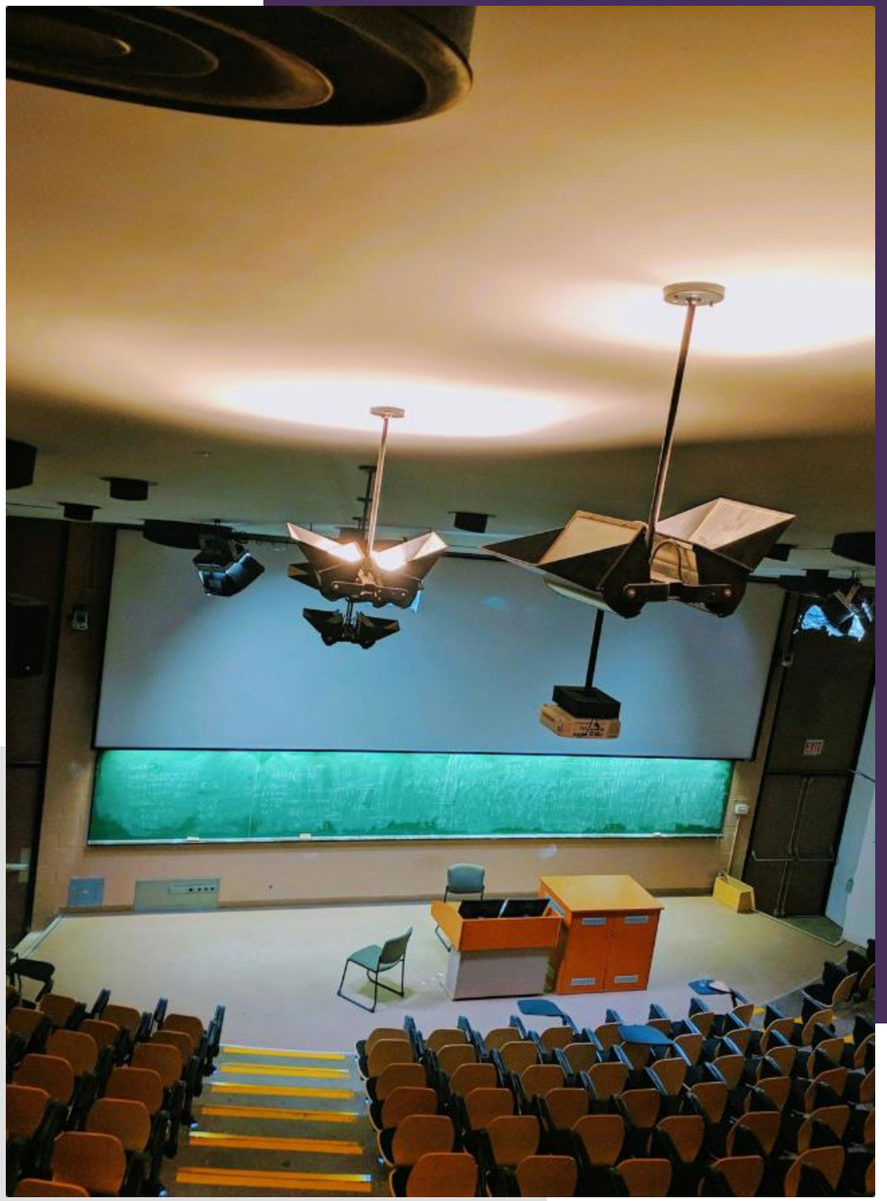


Continual Learning and Test in Production

How do we adapt our models to data distribution shifts? — Course: Advanced Machine Learning Systems; Audience: Upper-Level Undergraduate & Graduate Students

Duangdao Wichadakul

Lecture Agenda: Key Topics to Guide Today

Overview sections cover in continual learning and test in production

1. Part 1: Continual Learning
2. Part 2: Test in Production
3. Part 3: Summary

Lecture Agenda: Key Topics to Guide Today

Overview sections cover in continual learning and test in production

1. Part 1: Continual Learning
2. Part 2: Test in Production
3. Part 3: Summary

What is Continual Learning?

Continuously adapting models to distribution shifts

Goal of continual learning is to safely and efficiently automate the update.

All concepts here allows us design an ML system that is maintainable and adaptable to changing environments.

1 **Continual learning updates models to adapt to data distribution shifts**

Ongoing adaptation as data evolves

2 **Primarily an infrastructural problem**

Systems and pipelines enable safe updates

3 **Not about updating on every incoming sample due to catastrophic forgetting and too expensive**

Frequent single-sample updates can erase prior knowledge

4 **Models are updated in micro-batches (e.g., every 512 or 1,024 examples)**

Periodic updates using small batches of examples

What is **Continual Learning?**

A Simplified workflow of continual learning in a production

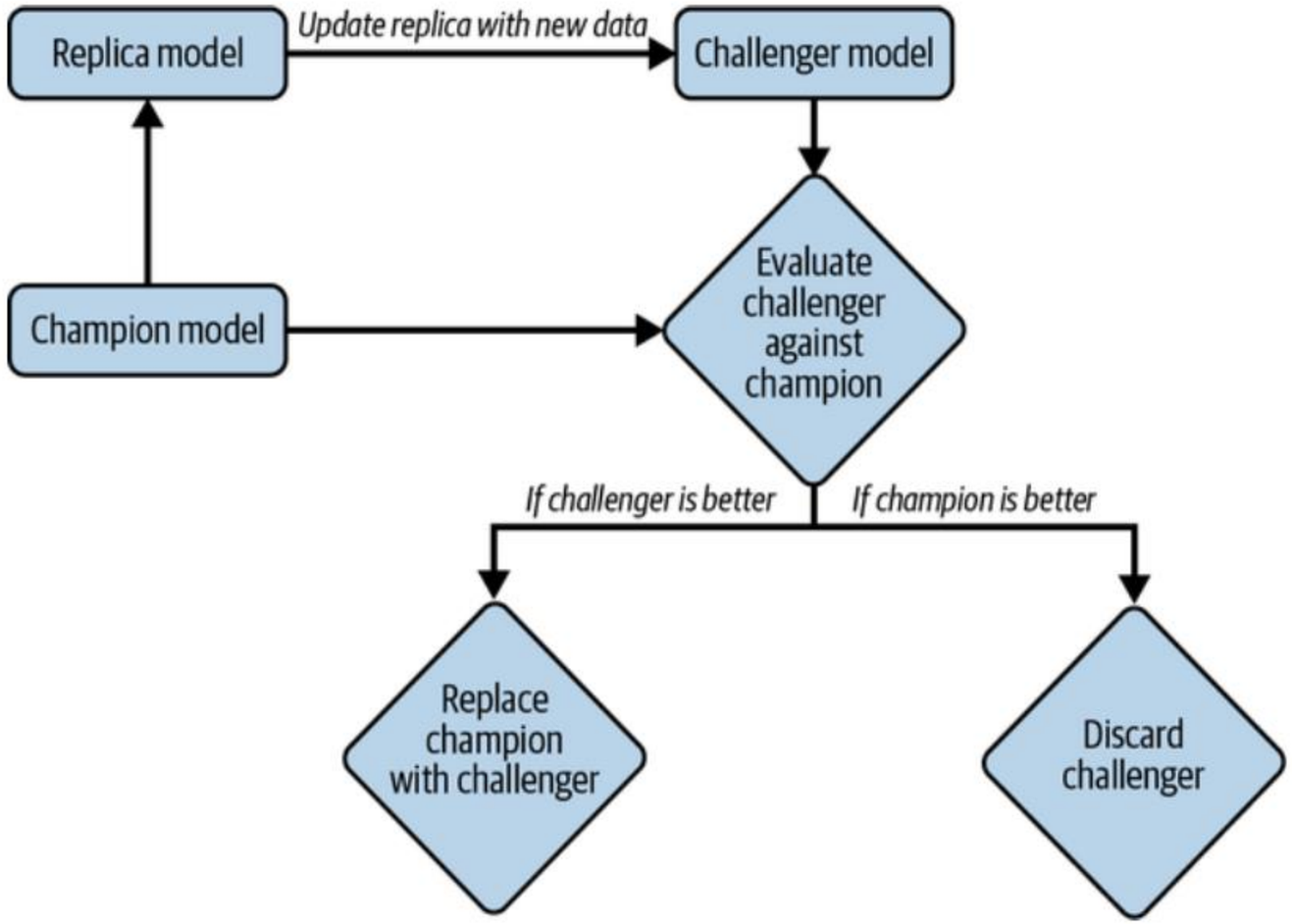


Figure 9-1. A simplification of how continual learning might work in production. In reality, the process of handling the failed challenger is a lot more sophisticated than simply discarding it.

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

Stateless Retraining versus **Stateful Training**

Compare training approaches: from scratch versus continuing from a prior model

Stateless Retraining

- Model trained from scratch each time
- Requires full dataset for each update
- No carryover of prior weights or state

Stateful Training

- Continues training from a previous state
- Also known as fine-tuning or incremental learning
- Updates with less data and often converges faster

Mainly applied to data iteration not model iteration

Stateless Retraining versus **Stateful Training**

Compare training approaches: from scratch versus continuing from a prior model

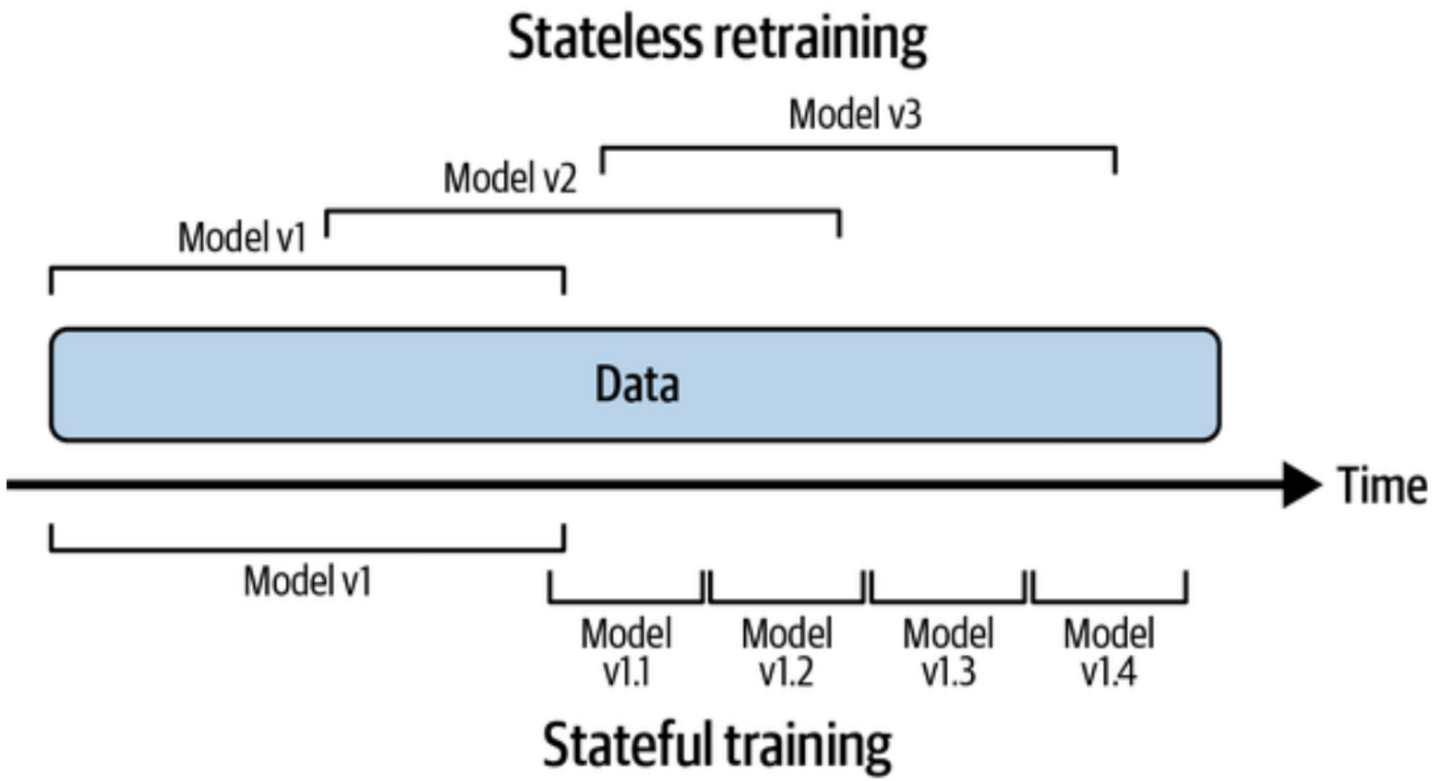


Figure 9-2. Stateless retraining versus stateful training

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

Why Continual Learning?

When models must adapt to shifts, rare events, and cold start

Continual learning is about setting up infrastructure that allows data scientists, ML engineers, to update the models whenever it is needed and to deploy this update quickly.

Cold start, a fundamental issue in recommendation system.

To combat data distribution shifts

Especially when shifts happen suddenly

Ex. Ride-sharing example: adapt to sudden surge in demand

Adjust prices and mobilize drivers quickly.

There is a specific event nearby and need more ride to that place.

To adapt to rare events with scarce historical data

Learn from limited examples during uncommon events

Ex. E-commerce example: learn customer behavior on Black Friday

Once-a-year event with unique patterns

To solve the continuous cold start problem for new or infrequent

Improve personalization despite sparse history
Also apply to existing users with different behaviors using different devices, e.g., notebook and mobile phones

Why Continual Learning?

When models must adapt to shifts, rare events, and cold start

Continual learning is about setting up infrastructure that allows data scientists, ML engineers, to update the models whenever it is needed and to deploy this update quickly.

To solve the continuous cold start problem for new or infrequent (cont.)

Coveo, a company that provides search engine and recommender systems to ecommerce websites find that it is common for an ecommerce site to have more than 70% of their shoppers visit their site less than three times a year.

Hene, the model must adapt quickly if possible during the visit to recommend what the users want.

The succes case is TikTok https://towardsdatascience.com/how-does-tiktoks-algorithm-know-me-so-well-459e8dc7e90b/

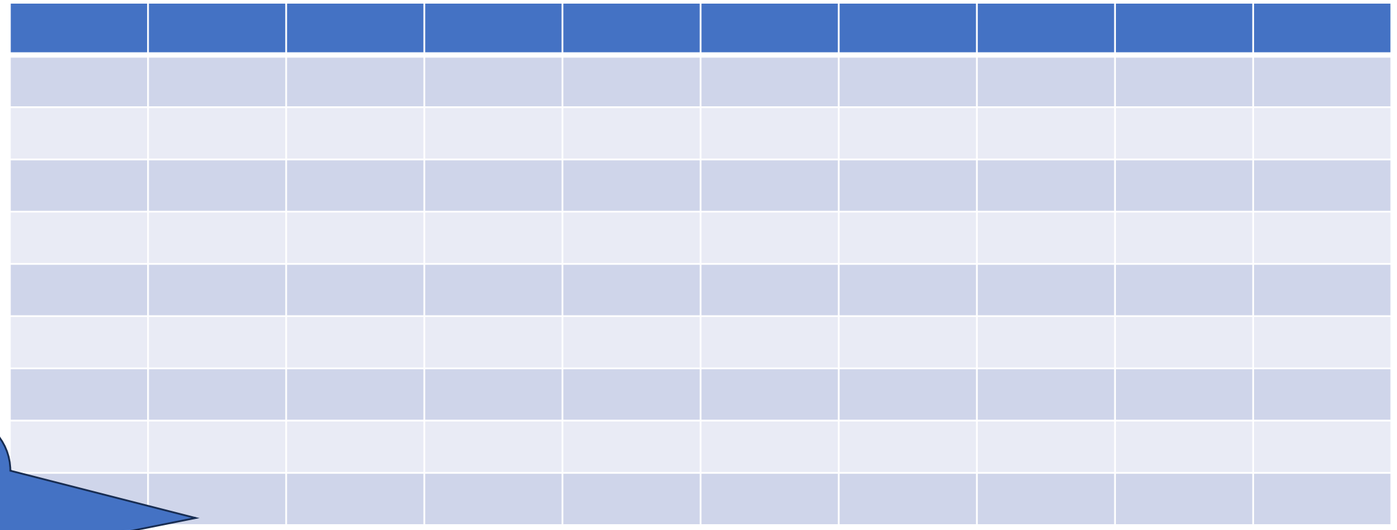
อ่านแล้วสรุปมาใน 2-3 ประโยค

Why Continual Learning?

When models must adapt to shifts, rare events, and cold start

Tiktok's main advantages

User-item matrix



Granularity of the feedback refers to how much information we get from it

Time a user spends on each video

Granular implicit feedbacks w time-based scale

Continual Learning Challenges

1. Fresh data access challenge
2. Evaluation challenge
3. Algorithm challenge

Challenge #1: Fresh Data Access for Frequent Model Updates

Challenges of real-time ingestion, labeling, and label computation

Label เช่น likes, favorites, clicks on videos

Updating Models Requires Continuous **fresh Data**

Models Need Ongoing Data To Stay Current

Warehouse Pulls Can Be Slow; Use **Kafka** Or **Kinesis** For Real-time Transport

Real-time Transports Reduce Ingestion Latency

Labeled Data Availability Often Bottlenecks Updates

Labeling Speed Limits Retraining Cadence

Extracting Labels From User Feedback Is Called **label Computation**

Compute Labels From Logs And Feedback Streams

Challenge #1: Fresh Data Access for Frequent Model Updates

Challenges of real-time ingestion, labeling, and label computation

LIVE

Warehouse Pulls Can Be Slow; Use **Kafka** Or **Kinesis** For Real-time Transport

Real-time Transports Reduce Ingestion Latency

ดึงข้อมูลจาก RT transport
โดยตรงทำให้ได้ข้อมูลใหม่ เร็วกว่า
ดึงจาก data warehouse

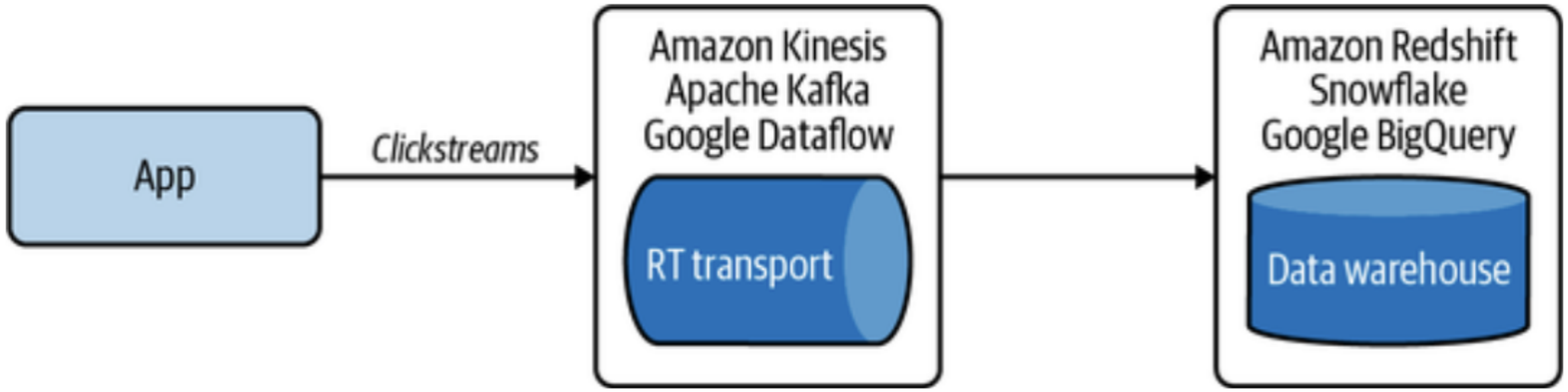


Figure 9-3. Pulling data directly from real-time transports, before it's deposited into data warehouses, can allow you to access fresher data

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

Challenge #1: Fresh Data Access for Frequent Model Updates

Challenges of real-time ingestion, labeling, and label computation

Extracting Labels From User Feedback Is Called **label Computation**

Compute Labels From Logs And Feedback Streams

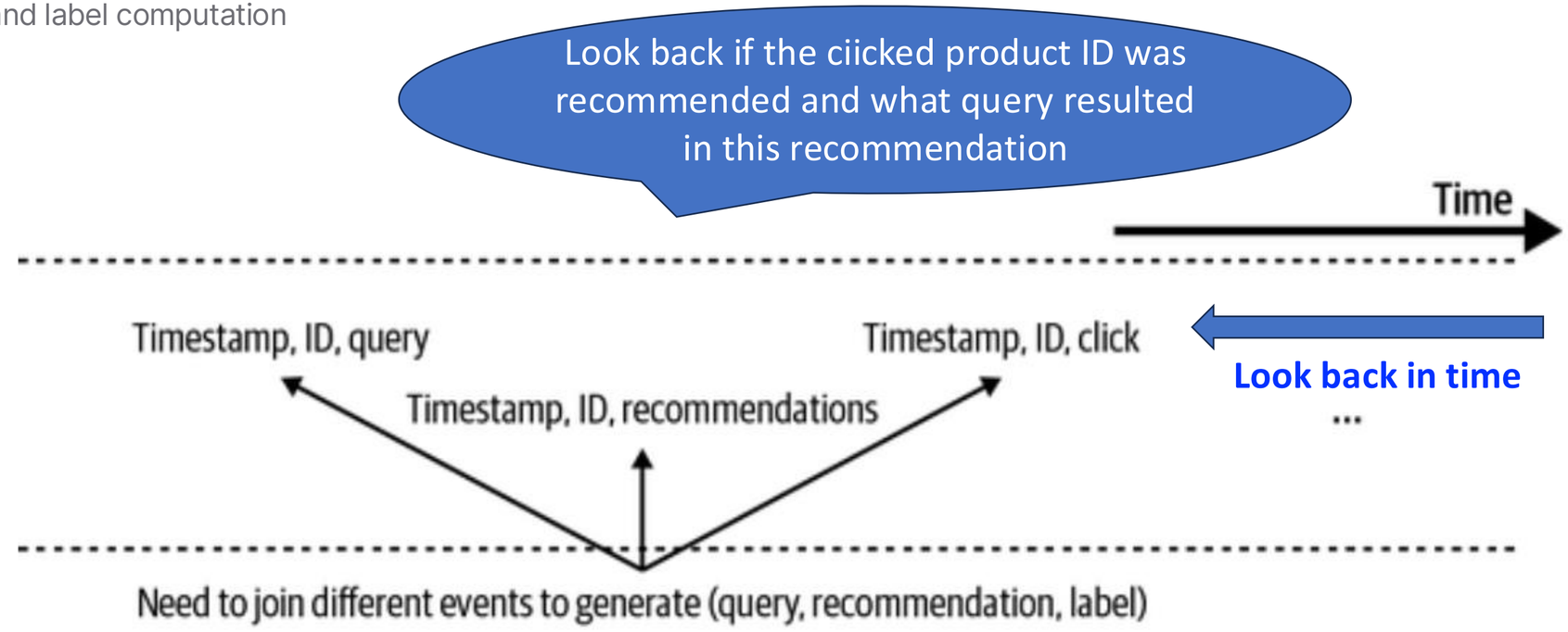


Figure 9-4. A simplification of the process of extracting labels from user feedback

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

Challenge #2: Evaluation Challenge

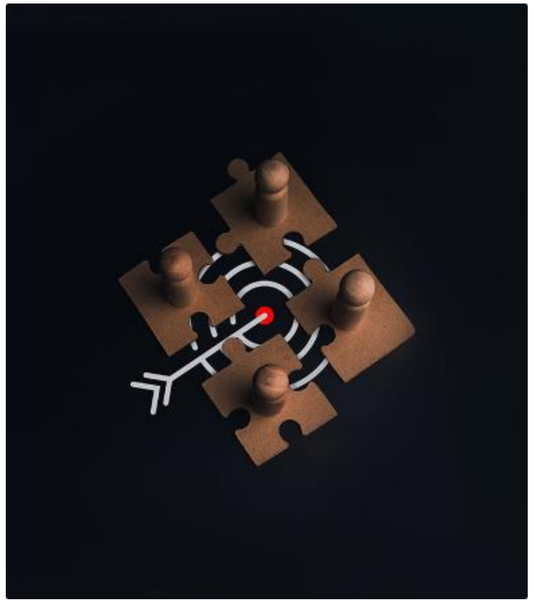
The biggest challenge is in making sure that this update is good enough to be deployed.

The risk of catastrophic failures amplifies with frequent updates.

Continual learning models are more susceptible to coordinated manipulation and adversarial attacks. Case study: Microsoft's chatbot, Tay, was shut down in 16 hours after trolls taught it to post offensive content.

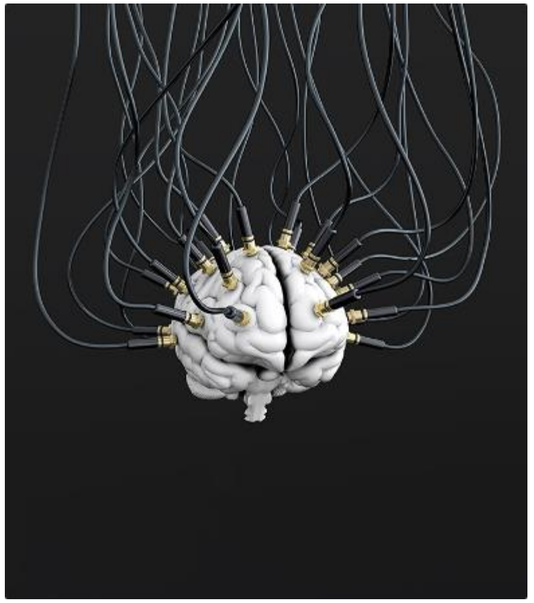
Evaluation takes time. Ex. The fraud patterns change quickly. Even though we want to update the model quickly, we need to perform A/B testing against the current model. Hence, must wait for some frauds come in.

Challenge #3: **Algorithm Challenges**



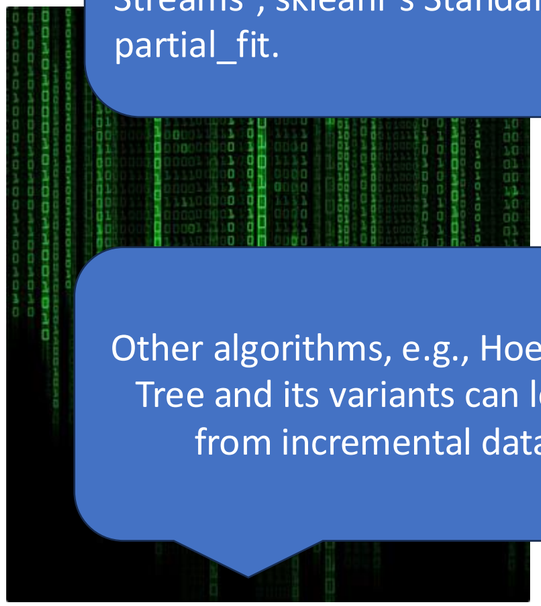
A softer challenge that affects certain algorithms more than others

Overview of differential impact on model families



Neural networks can be easily updated with data batches of any size

Supports incremental updates using mini-batches



Approximate these statistics incrementally for the new data, e.g., “Optimal Quantile Approximation in Streams”, skleanr’s StandardScaler partial_fit.

Other algorithms, e.g., Hoeffding Tree and its variants can learn from incremental data

Matrix-based (e.g., collaborative filtering) and tree-based models can be slow and expensive to update frequently

May require access to the entire dataset for retraining

Feature scaling statistics (min, max, variance) must be computed online to remain stable across data subsets

Use online algorithms for robust scaling metrics

The Four Stages of **Continual Learning**

A linear progression from manual retraining to production continual learning

Stage 1: Manual, stateless retraining

Ad hoc retraining done manually; no retained state between runs

Stage 2: Automated, stateless retraining

Scheduled or triggered retraining pipelines without preserved model state

Stage 3: Automated, stateful training

Training preserves state across runs to accelerate learning and adaptation

Stage 4: Continual learning

Ongoing learning from streaming data with continual model updates

Stage 1: Manual, Stateless Retraining

Manual, ad-hoc retraining when performance degrades

Process: Manual And Ad-hoc. A Data Engineer Queries Data, Cleans It, And Retrains A Model From Scratch.

Hands-on Retraining By Engineer; No Automation

Trigger: Done Only When Model Performance Has Severely Degraded.

Reactive Update Only After Major Degradation

Characteristics: Updating Existing Models Is A Low Priority. The Process Is Slow, Inefficient, And Prone To Bugs.

Low Priority, Slow, Inefficient, Bug Prone

Stage 2: Automated Retraining

Periodic batch retraining with scheduled automation

Process: A script automatically executes all retraining steps, run periodically by a batch process

Full retraining pipeline executed by an automated script on a schedule

Trigger: Retraining frequency is often based on a fixed schedule or gut feeling (for example, once a day)

Triggers commonly include fixed schedules or manual intuition such as daily runs

Requirements: Requires a scheduler (for example, Airflow), accessible data, and a model store (for example, MLflow, Amazon SageMaker)

Key components include a scheduler, reliable data access, and a model registry

Stage 3: Automated, **Stateful** Training

Scheduled fine-tuning from a prior checkpoint; reproducibility focus

Process: The automated script is reconfigured to perform stateful training (fine-tuning from a previous checkpoint).

Automated fine-tuning using prior checkpoints for continuity.

Trigger: Still based on a fixed schedule.

Scheduled runs remain the activation mechanism.

Requirements: Shift mindset from retraining from scratch; track data and model lineage for reproducibility and debugging.

Lineage tracking is critical for debugging and reproducibility.

Need model versioning

Stage 4: Continual **Learning**

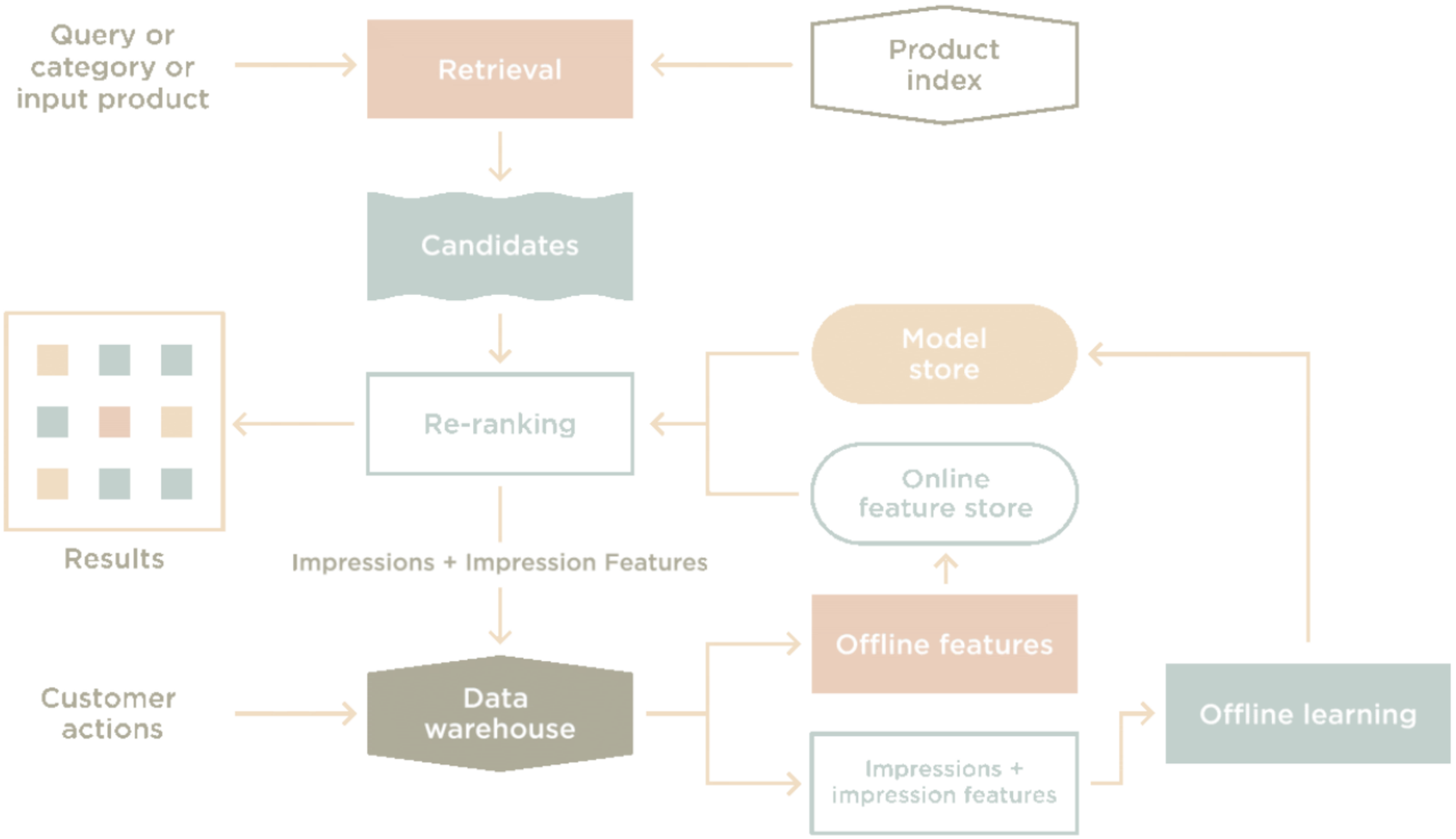
Automated updates driven by measurable triggers

Process: The models are automatically updated based on triggers, not a fixed schedule

Triggers: Automatically update the models based various measured aspects.

1- Time-based: ex. Every 5 minutes
2- Performance-based ex. model performance degrade
3- Volume-based: ex. Label data increases by 5%
4- Drift-based ex. Data distribution changes significantly

Requirements: Need a solid pipeline to continually evaluate the model updates to ensure good performance.



https://craft.faire.com/building-faires-new-marketplace-ranking-infrastructure-a53bf938aba0

Feature Reuse (Log and Wait)

Features extraced for model inference also logged for model retraining.

It saves computation and allows for consistency between prediction and training.
https://craft.faire.com/building-faires-new-marketplace-ranking-infrastructure-a53bf938aba0

https://www.geeksforgeeks.org/machine-learning/online-vs-offline-feature-store-understanding-the-differences-and-use-cases/

How Often to Update Your Models?

Weigh data freshness value against iteration costs

The answer depends on the value of data freshness

You must determine how much performance gain your model gets from being updated with fresh data

This is a trade-off between **model iteration** (finding a better architecture) and **data iteration** (refreshing with new data)

How Often to Update Your Models?

Weigh data freshness value against iteration costs

Measure how performance changes for different time-window

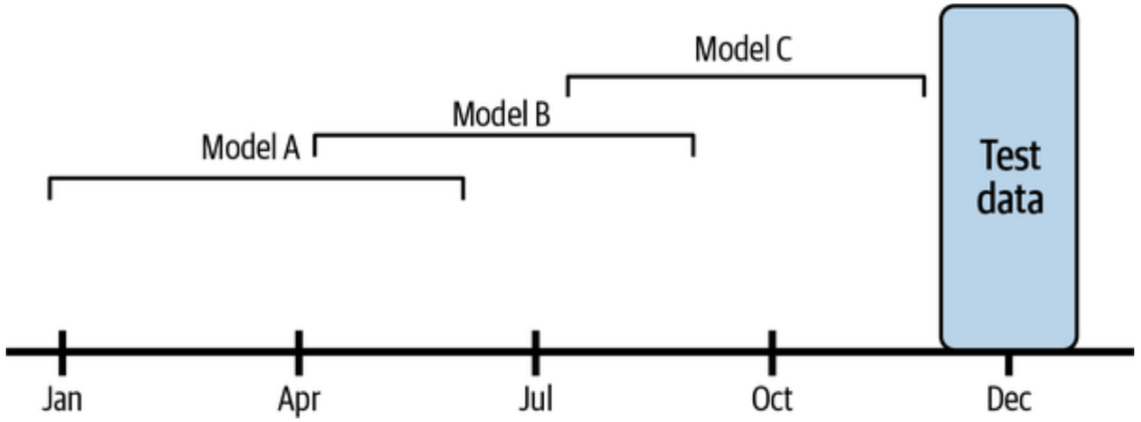


Figure 9-5. To get a sense of the performance gain you can get from fresher data, train your model on data from different time windows in the past and test on data from today to see how the performance changes

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

In 2014, Facebook experimentd in ad click-through-rate prediction and found out that they could reduce the model's loss by 1% by going from retraining weekly to retraining dialy.

It depends on our data and experiments

How Often to Update Your Models?

Weigh data freshness value against iteration costs

It is the trade-off between the computing resource + budget we need and the performance we gain.

It depends on our data and experiments

Model Iteration and Data Iteration

Lecture Agenda: Key Topics to Guide Today

Overview sections cover in continual learning and test in production

1. Part 1: Continual Learning
2. Part 2: Test in Production
3. Part 3: Summary

Part 2

Part 2: Test in Production

Offline evaluation is not enough; evaluate models with live data safely

Why Test in Production?

With data distribution changes, even though the model does well on data from the last hour (via a backtest), does not guarantee it will continue doing well on the future data.

The only way to know whether a model will do well in production in to deploy it.

Test in production is terrifying but necessary

Test in Production: Key Techniques

Practical methods for evaluating your models in production safely

1
Shadow Deployment

2
A/B Testing

3
Canary Release

4
Interleaving Experiments

5
Bandits

1 Shadow Deployment

might be the safest way to deploy your model

1. Deploy the candidate model in parallel with the existing model.

2. For each incoming request, route it to both models to make predictions, but only serve the existing model's prediction to the users.

3. Log the predictions from the new model for analysis purposes.

Seems to be safest but expensive.

2 A/B Testing: Compare Two Variants

Measure effectiveness with randomized traffic and sufficient samples

Concept: Compare two variants (e.g., model A vs. model B) by routing users to each and measuring which is more effective

Requirement: Requires true randomization of traffic and a sufficient number of samples

Execution: A percentage of traffic is routed to the new model, and the rest to the old one

Analysis: Monitor predictions and user feedback to determine if the performance difference is statistically significant

How can we determine the number of samples sufficient to conclue that a model is better then the other

2 A/B Testing: Compare Two Variants

Measure effectiveness with randomized traffic and sufficient samples

1. **Baseline Conversion Rate (BCR)**
   The performance of the current version (e.g., A)

2. **Minimum Detectable Effect (MDE)**
   The smallest improvement we want to be able to detect.

3. **Statistical Significant (Alpha)**
   This the probability of false positive allows. If 95% significance, it means, we accept 5% change of false positives.

4. **Statistical Power (1-Beta)**
   The probability of detecting a real effect. If there is a real difference, what's the chance our test will be able to detect it.

How can we determine the number of samples sufficient to conclue that a model is better then the other

2 A/B Testing: Compare Two Variants

Measure effectiveness with randomized traffic and sufficient samples

- A/B testing is a standard method for testing models in production
- A/B testing is stateless. We just random the traffic without the knowing the current models' performance
- We can do A/B testing even with the batch prediction

We can also do A/B/C or A/B/C/D testing

3 Canary Release: Controlled Rollout

Reduce release risk by routing limited traffic to a new model

Concept

Reduce risk by slowly rolling out the new release to a small subset of users

Process

If performance is satisfactory, increase traffic gradually; if degraded, abort canary and route all traffic back to the existing model

Execution

Deploy the candidate model called the canary and route a small portion of traffic to it

Read: **Automated Canary Analysis at Netflix with Kayenta**
https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69

4 Interleaving Experiments: Evaluate Competing Models

Expose one user to mixed recommendations from multiple models for click-based comparison

originally proposed by Thorsten Joachims in 2002
for the problems of search rankings.

Netflix found that interleaving “reliably identifies the best algorithms with considerably smaller sample size compared to traditional A/B testing”

Concept

Expose a single user to recommendations from multiple models simultaneously.

Execution

Recommendations from model A and model B are interleaved in the user’s results instead of splitting users into groups.

Use Case

Highly effective for evaluating recommender and search ranking systems.

Analysis

Determine the winning model by which model’s recommendations the user clicks on more.

4 Interleaving Experiments: Evaluate Competing Models

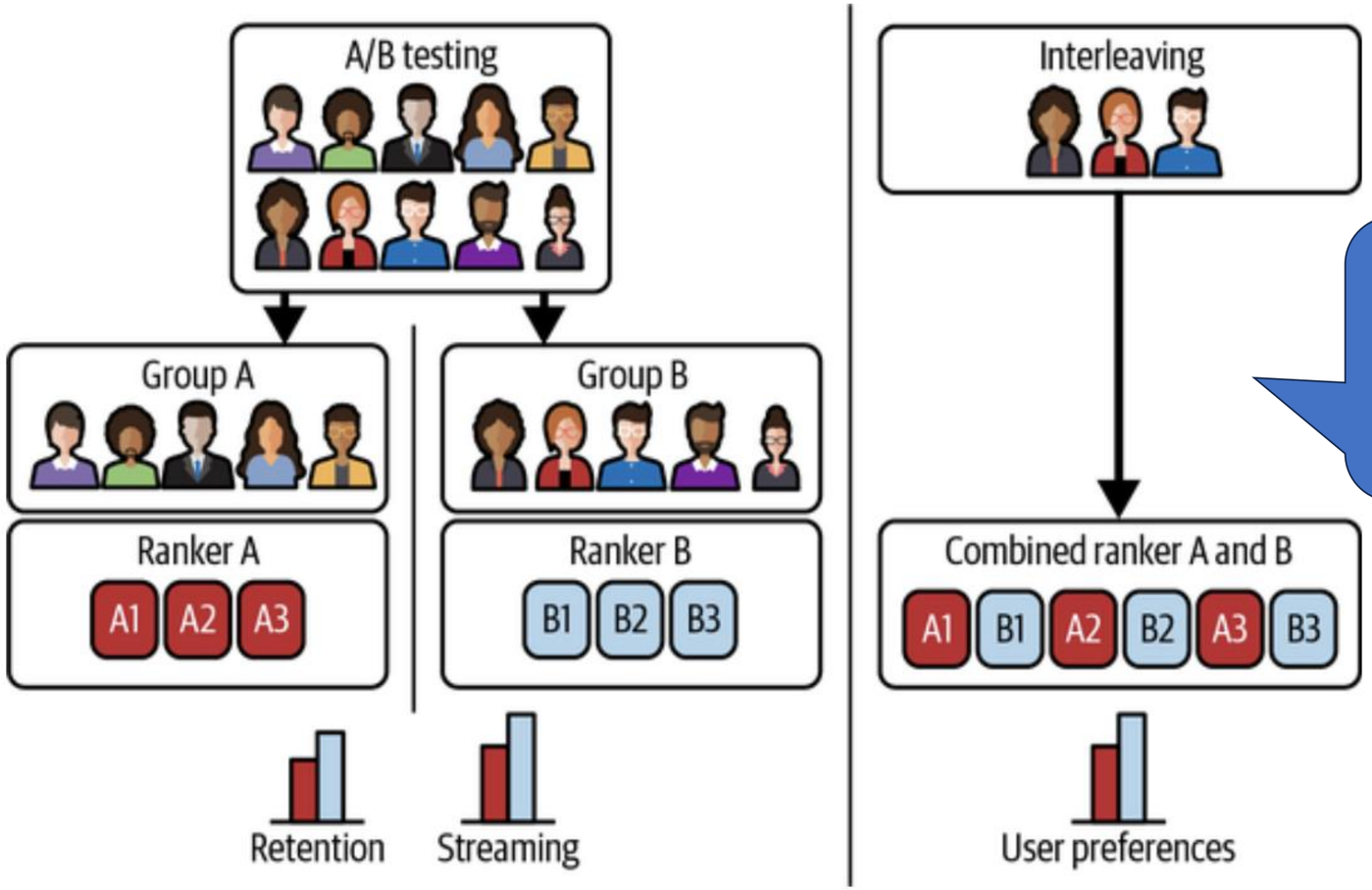


Figure 9-6. An illustration of interleaving versus A/B testing. Source: Adapted from an image by Parks et al.

Must ensure that positions of buttons won't affect the clicks among Rankers A and B.

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

4 Interleaving Experiments: Evaluate Competing Models

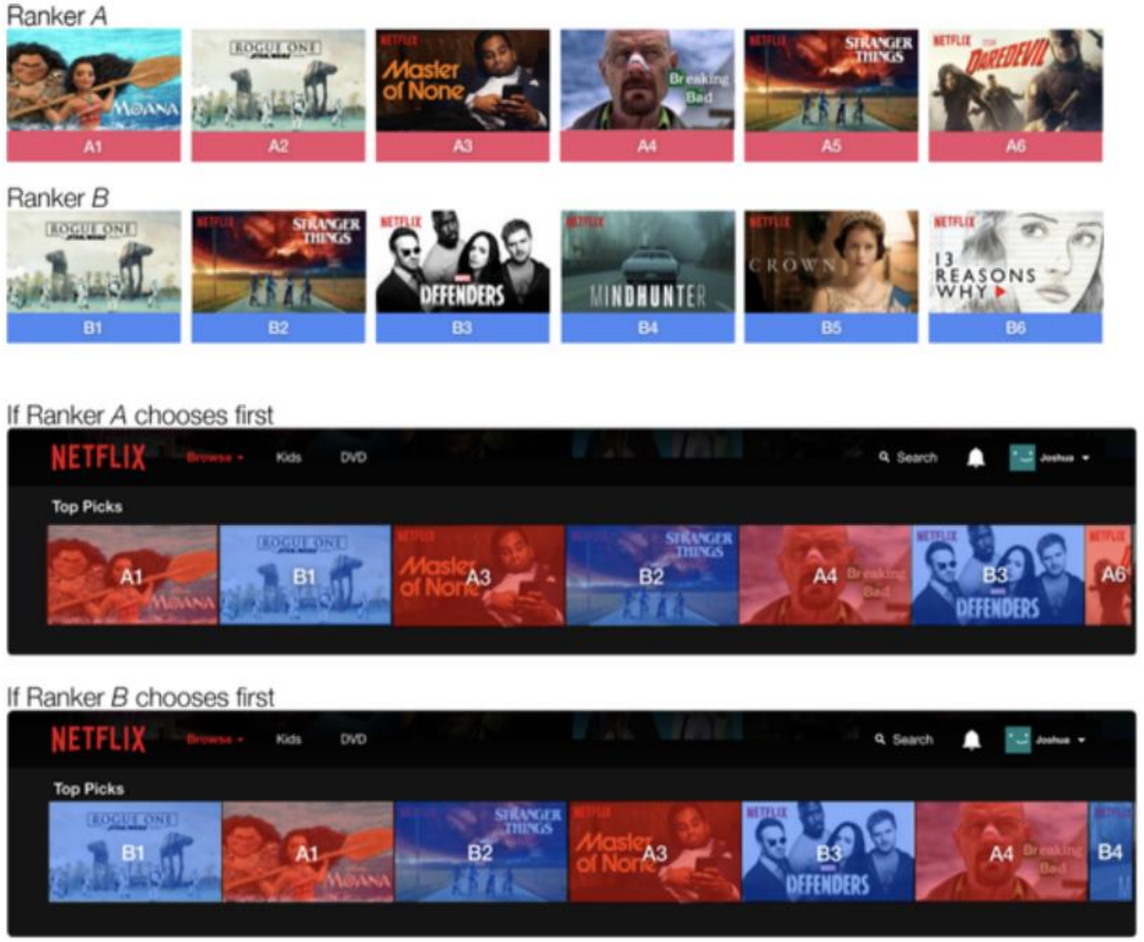


Figure 9-7. Interleaving video recommendations from two ranking algorithms using team draft. Source: Parks et al.[32]

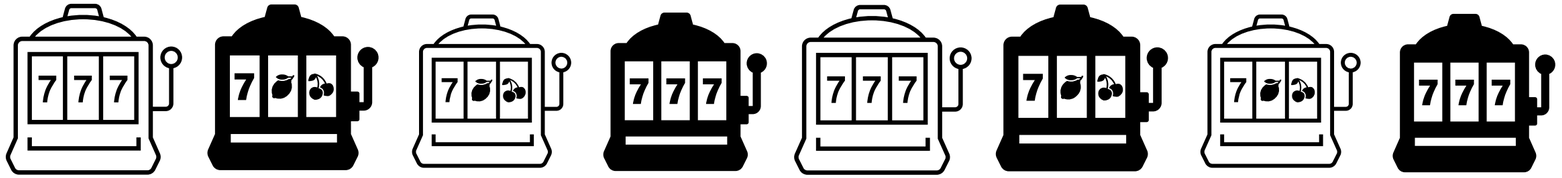
Must ensure that positions of buttons won't affect the clicks among Rankers A and B.

Source: Chip Huyen, Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. Chapter 9. O'Reilly Media, Inc. 2022

5 Bandits: Balancing **Exploitation** and Exploration

Stateful routing that learns and shifts traffic to better models

Bandit algorithms were originated in gambling



A slot machine or one-armed bandit

Experiment over time to find out which machine maximixe the payout.

Multi-armed bandits are algorithms that allow you to balance between exploitation (choosing the slot machine that has paid the most in the past) and **exploration** (choosing other slot machines that may pay off even more).

Exploitation

Use the best-performing model so far

Exploration

Try other models that might be better

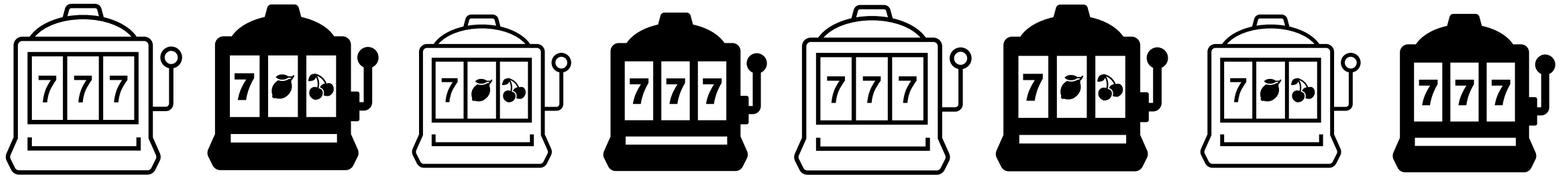
Data efficiency

More data-efficient than traditional A/B testing

5 Bandits: Balancing **Exploitation** and Exploration

Stateful routing that learns and shifts traffic to better models

Bandit algorithms were originated in gambling



A slot machine or one-armed bandit

- Bandits allow you to determine how to route traffic to each model for prediction to determine the best model while maximizing prediction accuracy for your users

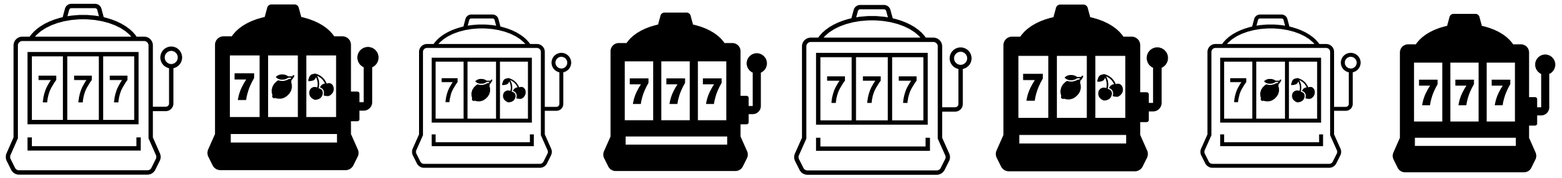
- Bandits is stateful: before routing, we need to calculate the all models' current performance which requires:
  1. Model must be able to make online predictions
  2. Preferably short feedback loops
  3. A mechanism to collect feedback, calculate and keep track of each mode's performance and route prediction request accordingly

- Bandits are well-studied in academia and have been shown to be a lot more data-efficient than A/B testing

In an experiment by Google's Greg Rafferty, A/B testing required over 630,000 samples to get a confidence interval of 95%, whereas a simple bandit algorithm (Thompson Sampling) determined that a model was 5% better than the other with less than 12,000 samples.

5 Bandits Algorithms

ε-greedy

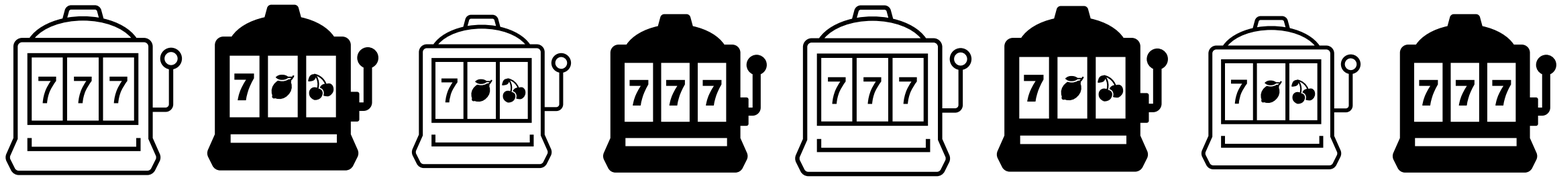


A slot machine or one-armed bandit

- The simplest algorithm for exploration is ε-greedy
  - If ε=0.9, it means 90% of the time, the request will be routed to the model with the best performance
- Two of the most popular exploration algorithms are
  - Thomson Sampling
  - Upper Confidence Bound (UCB)

5 Bandits Algorithms

Thomson Sampling



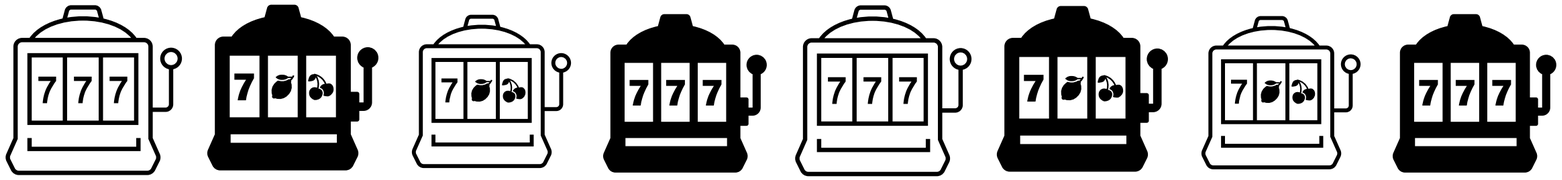
A slot machine or one-armed bandit

- Thomson Sampling is a Bayesian approach using prior knowledge to estimate the expected reward of each action
- The algorithm maintains a beta probability distribution over the expected reward of each action.
- It selects the action with the highest expected reward based on a random draw from the distribution.
- It updates the probability distribution based on the observed reward.

https://medium.com/@reddyyashu20/what-is-thompson-sampling-and-upper-confidence-bound-d8088a703b02

5 Bandits Algorithms

Thomson Sampling



A slot machine or one-armed bandit

```
def thompson_sampling(n_arms, n_rounds, get_reward_func):
    # initialize parameters
    alpha = np.ones(n_arms)
    beta = np.ones(n_arms)
    rewards = np.zeros(n_arms)

    for i in range(n_rounds):
        # sample a random reward for each arm based on the current
distribution
        samples = [np.random.beta(alpha[j], beta[j]) for j in
range(n_arms)]

        # select the arm with the highest sample
        arm = np.argmax(samples)

        # get the reward for the selected arm
        reward = get_reward_func(arm)

        # update the distribution for the selected arm
        alpha[arm] += reward
        beta[arm] += 1 - reward

        # update the total rewards for each arm
        rewards[arm] += reward

    return rewards
```

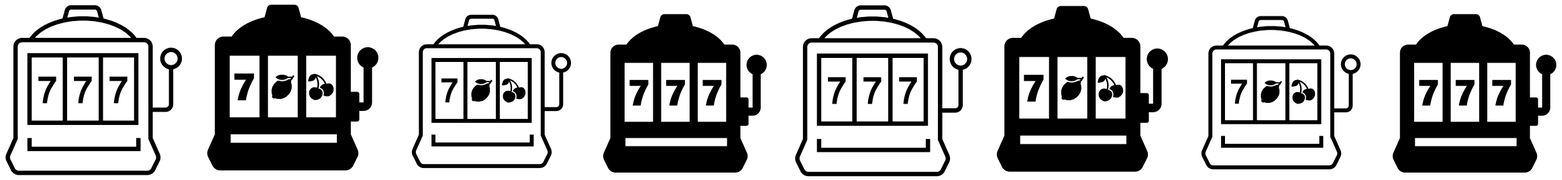
- n_arms is the number of actions
- n_rounds is the numbef of times the algorithm will select an action
- get_reward_function is a function that takes an arm as input and returns the reward for that arm

Model beliefs with beta distribution

https://medium.com/@reddyyashu20/what-is-thompson-sampling-and-upper-confidence-bound-d8088a703b02

5 Bandits Algorithms

Thomson Sampling: intuition



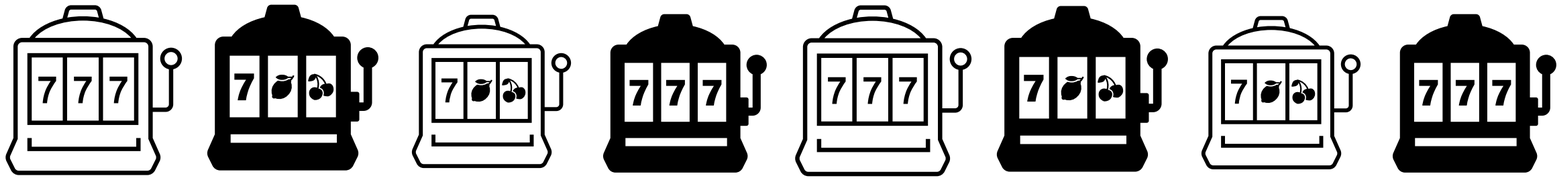
A slot machine or one-armed bandit

An arm that has been tried many times and has a good track record will have a "narrow and tall" probability distribution centered on a high value. It will consistently produce high samples.

An arm that is new or has been tried very few times will have a "wide and flat" distribution, representing high uncertainty. Because the distribution is wide, it has a chance (even if small) of producing a very high random sample, which gives it a chance to be explored.

5 Bandits Algorithms

Upper Confidence Bound (UCB)



A slot machine or one-armed bandit

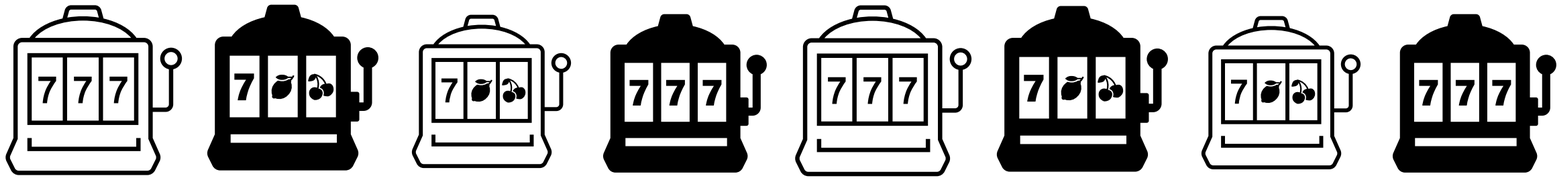
- Upper Confidence Bound (UCB) balances the exploitation and exploration by selecting the action with the highest upper confidence bound.
- The upper confidence bound measures how uncertain the expected reward estimate is for each action.
- Actions with high uncertainty are given higher priority to explore.

https://medium.com/@reddyyashu20/what-is-thompson-sampling-and-upper-confidence-bound-d8088a703b02

5 Bandits Algorithms

Upper Confidence Bound (UCB)

- n_arms is the number of actions
- n_rounds is the numbef of times the algorithm will select an action
- get_reward_function is a function that takes an arm as input and returns the reward for that arm
- c is a constant that control the balance between the exploitation and exploration. The higher the value of c, the more the algorithm will explore.



A slot machine or one-armed bandit

```
def ucb(n_arms, n_rounds, get_reward_func, c=1.0):
    # initialize parameters
    rewards = np.zeros(n_arms)
    n_selections = np.zeros(n_arms)

    for i in range(n_rounds):
        # calculate the upper confidence bound for each arm
        ucb_values = rewards + c * np.sqrt(np.log(i+1) /
(n_selections + 1e-6))

        # select the arm with the highest upper confidence bound
        arm = np.argmax(ucb_values)

        # get the reward for the selected arm
        reward = get_reward_func(arm)

        # update the total rewards and number of selections for the
selected arm
        rewards[arm] += reward
        n_selections[arm] += 1

    return rewards
```

$$\mathrm{UCB}(a) = X_a + \sqrt{\frac{2 \ln t}{n_a}}$$

optimism in the face of uncertainty

https://medium.com/@reddyyashu20/what-is-thompson-sampling-and-upper-confidence-bound-d8088a703b02

Contextual **Bandits**

An an exploration strategy to determine the payout of each prediction (not the overall model predictions)

Contextual bandits (also called one-shot reinforcement learning) are algorithms that help you balance between showing users the items they will like and showing the items that you want feedback on.

Concept: An Exploration Strategy To Determine The Payout Of Each Individual **action** (e.g., Which Specific Item To Recommend), Not Just Each Model.

Targets Per-action Outcomes Rather Than Per-model Averages.

Challenge: Solves The partial Feedback Problem, Where You Only Get Feedback On The 10 Items You Showed, Not The 990 You Didn't.

Handles Sparse Feedback From Displayed Items Only.

Implementation: More Difficult To Implement Than Model Bandits Because The Exploration Strategy Depends On The Model's Architecture.

Requires Integration With Model Internals And Action Space.

Contextual **Bandits**

An an exploration strategy to determine the payout of each prediction (not the overall model predictions)

Deep Bayesian Bandits: Exploring in Online Personalized Recommendations

DALIN GUO, University of California, San Diego, USA
SOFIA IRA KTENA, Twitter, UK
FERENC HUSZAR, Twitter, UK
PRANAY KUMAR MYANA, Twitter, UK
WENZHE SHI, Twitter, UK
ALYKHAN TEJANI, Twitter, UK

Contextual bandits are well researched and have been shown to improve models' performance significantly, reported by Twitter in 2020.

Recommender systems trained in a continuous learning fashion are plagued by the feedback loop problem, also known as algorithmic bias. This causes a newly trained model to act greedily and favor items that have already been engaged by users. This behavior is particularly harmful in personalised ads recommendations, as it can also cause new campaigns to remain unexplored. Exploration aims to address this limitation by providing new information about the environment, which encompasses user preference, and can lead to higher long-term reward. In this work, we formulate a display advertising recommender as a contextual bandit and implement exploration techniques that require sampling from the posterior distribution of click-through-rates in a computationally tractable manner. Traditional large-scale deep learning models do not provide uncertainty estimates by default. We approximate these uncertainty measurements of the predictions by employing a bootstrapped model with multiple heads and dropout units. We benchmark a number of different models in an offline simulation environment using a publicly available dataset of user-ads engagements. We test our proposed deep Bayesian bandits algorithm in the offline simulation and online AB setting with large-scale production traffic, where we demonstrate a positive gain of our exploration model.

CCS Concepts: • **Computing methodologies** → **Reinforcement learning**; **Neural networks**; • **Information systems** → **Display advertising**.

Additional Key Words and Phrases: Recommender Systems, Algorithmic bias, Contextual bandit

https://arxiv.org/pdf/2008.00727

Contextual **Bandits**

An an exploration strategy to determine the payout of each prediction (not the overall model predictions)

Contextual bandits are well researched and have been shown to improve models' performance significantly, reported by Google in 2022.

AutoML for Contextual Bandits

Praneet Dutta, Joe Cheuk, Jonathan S Kim, Massimo Mascaro*
{praneetdutta,joecheuk,jonathanskim,massy}@google.com
Google Inc.
Mountain View, California

2 Feb 2022

ABSTRACT

Contextual Bandits is one of the widely popular techniques used in applications such as personalization, recommendation systems, mobile health, causal marketing etc [12]. As a dynamic approach, it can be more efficient than standard A/B testing in minimizing regret. We propose an end to end automated meta-learning pipeline to approximate the optimal Q function for contextual bandits problems. We see that our model is able to perform much better than random exploration, being more regret efficient and able to converge with a limited number of samples, while remaining very general and easy to use due to the meta-learning approach. We used a linearly annealed e-greedy exploration policy to define the exploration vs exploitation schedule. We tested the system on a synthetic environment to characterize it fully and we evaluated it on some open source datasets [2] to benchmark against prior work. We see that

given action in a given state:

$$Q(s, a) = E[r|s, a] \tag{1}$$

where r is the reward, s the state and a the action. Armed with an approximated Q-function generated by AutoML automatically, we can create easily an exploratory policy using the ϵ-greedy exploration schedule. The Q-function, and thus the policy, is updated periodically as new batches of data are accumulated. Being a function of the state space, the Q-function will be able to generalize across it. Note that we use here the Reinforcement Learning term "Q-function" even if the system we study is purely a contextual bandit one. We believe the notation and this work can be extended to some multiaction problems by replacing the immediate reward with a long term discounted reward to implement a simple form of Q-Learning. We plan to address this idea in a future work.

https://arxiv.org/pdf/1909.03212

Problem with current ML model evaluation

- The evaluation process is often handled by the same data scientists who developed the model.
- This introduces bias, as developers have a different context and usage pattern than end-users.
- Evaluation is frequently done ad-hoc, leading to inconsistent and variable results between different data scientists.
- This lack of a standardized process causes models to fail in production and creates anxiety for data scientists about deployment.

Proposed solution

- Teams should establish a clear, standardized evaluation pipeline.
- This pipeline should define which tests to run, in what order, and what thresholds a model must pass.
- The evaluation process should be automated, similar to CI/CD in software engineering, triggering automatically with each new model update.
- Evaluation results should be formally reported and reviewed.

Overview

Part 3: Summary

A review of the key concepts discussed in this lecture.

Summary: Continual Learning

Infrastructural approach for adapting models to changing data

Continual learning is primarily an infrastructural solution for adapting models to changing data distributions.

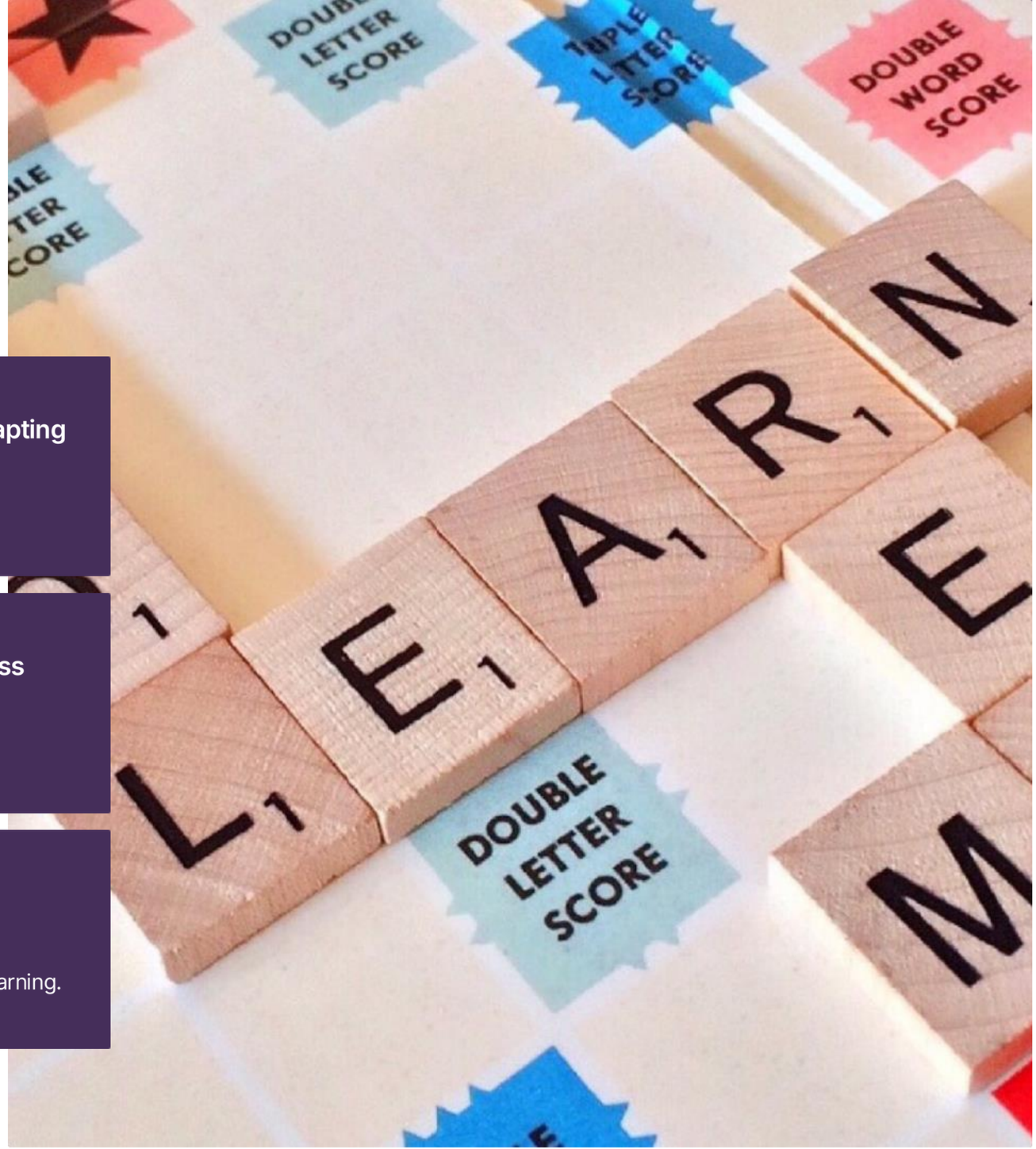
Focuses on the system-level capability to update models as data shifts.

Companies progress through four stages: from manual, stateless retraining to fully automated, trigger-based continual learning.

A maturity path from manual retraining to automated triggers and pipelines.

Continual learning requires mature streaming infrastructure, especially for online evaluation.

Real-time pipelines and online evaluation are essential to sustain continual learning.



Model Updates and Evaluation: Key Takeaways

Balancing data freshness, safe testing, and robust evaluation

1 Summary: Model Updates and Evaluation

- Answer update cadence by measuring the value of data freshness
- Safely test in production using shadow deployment to bandits
- Maintain a robust evaluation pipeline to manage update risks

2 Final Thoughts on Evaluation Process

- Define not just what tests to run but who runs them
- Automate evaluation pipelines and trigger on each model update
- Foster close collaboration between data science and platform teams